

OPEN UNIVERSITY OF SRI LANKA

Faculty of Natural Science

INDIVIDUAL CONTRIBUTION REPORT

BY GROUP LEADER

MINI PROJECT

(Group no:03)

OUSL STUDENT MAP

Submitted by:

H M Tharaka Sadharuwan

Group leader

Student ID: 124682005

Other Group members:

1. **O K Kalana Indusara : 724682892**
2. **A G D R Jayaweera : 624698181**
3. **G W G C B Sanjeewa : 524655985**
4. **I V C M Bandara : 724655849**
5. **K L K P Liyanage : 524679572**
6. **R S Warnakulasooriya : 624683193**

Course code: COU4303

Course name: Artificial Intelligence

Academic Year : 2024/2025

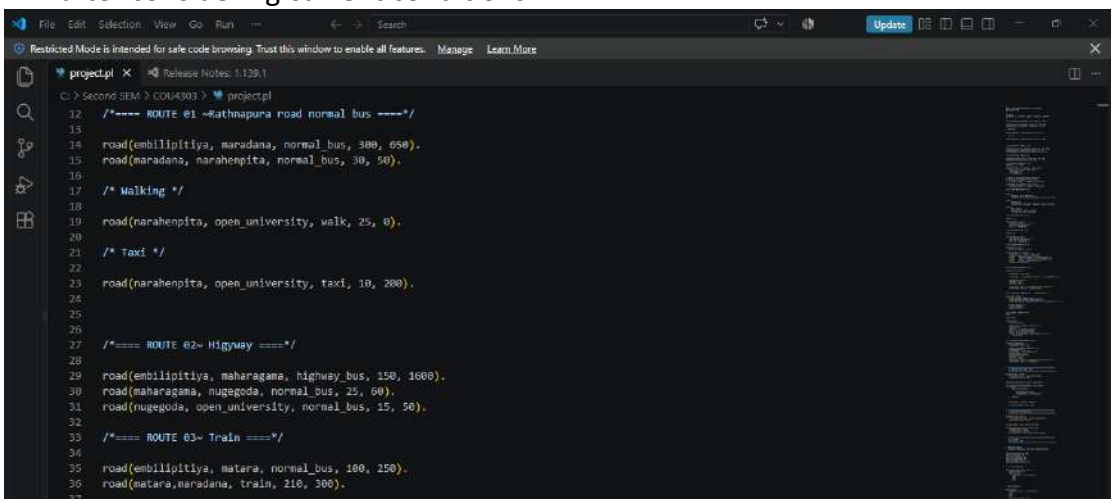
Introduction

We are developing a system for the convenience of students studying at the Open University.This system informs students—whether they are from the Colombo area or remote locations—about the routes available to reach the university, while also providing rough estimates regarding security, travel time, and costs for each route.Once students register at the university, we enter their location into this system using their address.

Contribution of each members

Group leader's contribution : 124682005

- Presenting a basic idea about the overall project and a rough outline for it.
- Coordinating the team and supervising the tasks assigned each members.
- Creating the basic structure for the code.
- Determining routes and inputting data into the code specially regarding transport mode costs and travel times for the path associated with each route after considering current conditions.



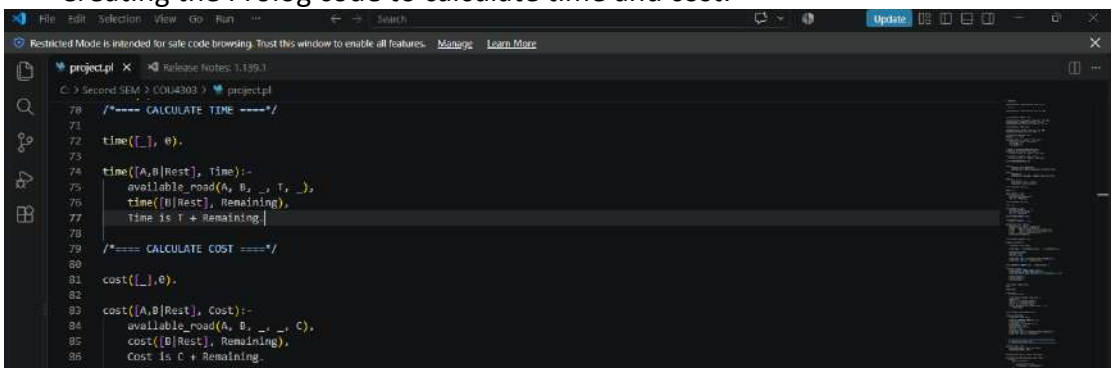
- Adding comments to the code for ease of reading and understanding.

- Code was developed to interconnect routes and enable users to travel to their destinations using various modes of transport and routes from a single stop, while simultaneously calculating the respective travel times and costs.



2. O K Kalana Indusara : 724682892

- Creating the Prolog code to calculate time and cost.



```
/*---- CALCULATE TIME ----*/

time([_], 0).

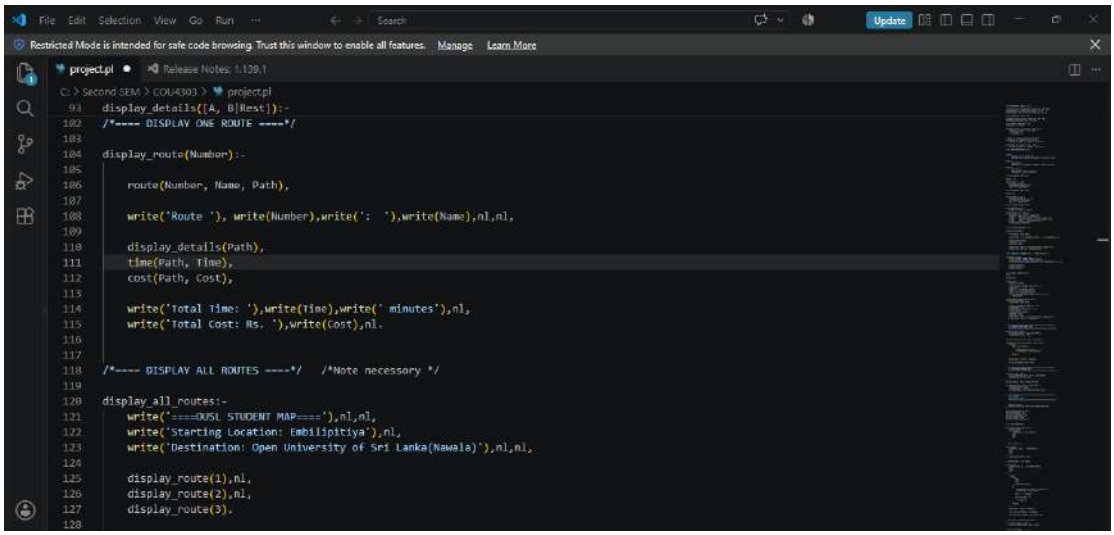
time([A,B|Rest], Time):-
    available_road(A, B, _, T, _),
    time([B|Rest], Remaining),
    Time is T + Remaining.

/*---- CALCULATE COST ----*/

cost([_],0).

cost([A,B|Rest], Cost):-
    available_road(A, B, _, _, C),
    cost([B|Rest], Remaining),
    Cost is C + Remaining.
```

- The code segment responsible for displaying the route, time, cost, and transport method for each route was created.



```
display_details([A, B|Rest]):-
/*---- DISPLAY ONE ROUTE ----*/

display_route(Number):-

    route(Number, Name, Path),

    write('Route '), write(Number),write(':  '),write(Name),nl,nl,

    display_details(Path),
    time(Path, Time),
    cost(Path, Cost),

    write('Total Time: '),write(Time),write(' minutes'),nl,
    write('Total Cost: Rs. '),write(Cost),nl.

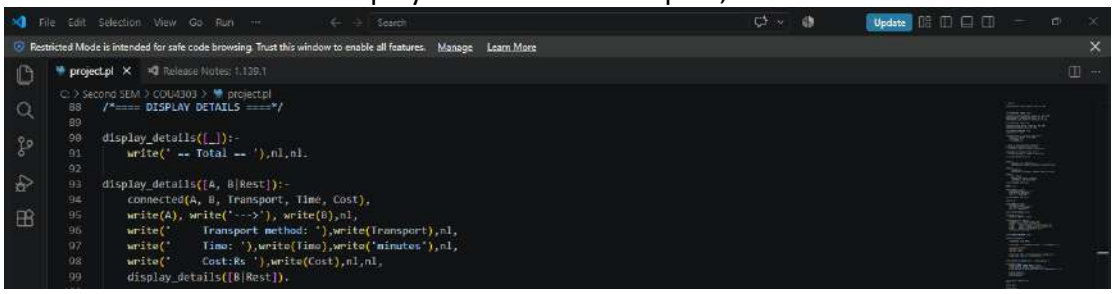

/*---- DISPLAY ALL ROUTES ----*/   /*Note necessory */

display_all_routes:-
    write('====OUSL STUDENT MAP===='),nl,nl,
    write('Starting Location: Embilipitiya'),nl,
    write('Destination: Open University of Sri Lanka(Nawala)'),nl,nl,

    display_route(1),nl,
    display_route(2),nl,
    display_route(3).
```

3. A G D R Jayaweera : 624698181

- Code was created to display the mode of transport, time and cost.

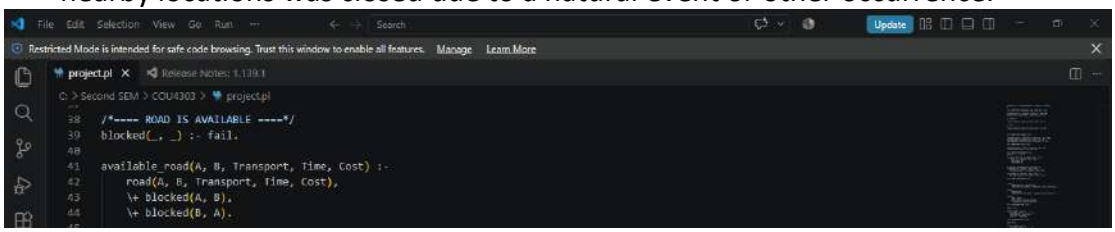


```
/*==== DISPLAY DETAILS ====*/

display_details([_]):-
    write(' -- Total -- '),nl,nl.

display_details([A, B|Rest]):-
    connected(A, B, Transport, Time, Cost),
    write(A), write('--->'), write(B),nl,
    write('     Transport method: '),write(Transport),nl,
    write('     Time: '),write(Time),write('minutes'),nl,
    write('     Cost:Rs '),write(Cost),nl,nl,
    display_details([B|Rest]).
```

- A blocked route was created to inform users when the road between two nearby locations was closed due to a natural event or other occurrence.

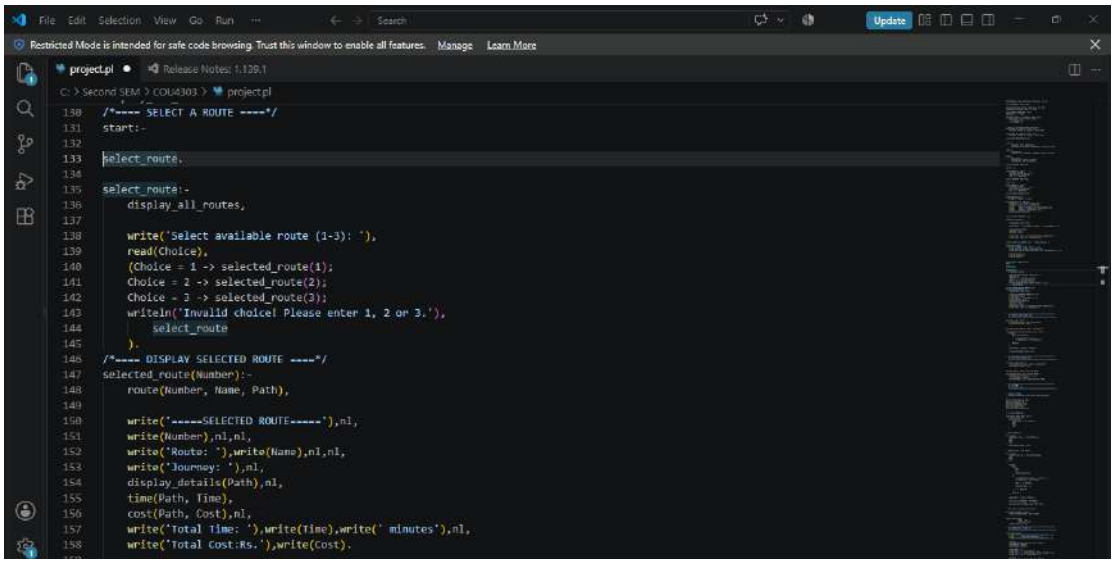


```
/*---- ROAD IS AVAILABLE ----*/
blocked(_, _) :- fail.

available_road(A, B, Transport, Time, Cost) :-
    road(A, B, Transport, Time, Cost),
    \+ blocked(A, B),
    \+ blocked(B, A).
```

4 . G W G C B Sanjeewa : 524655985

- Code was developed to display the transport modes, stops, travel time and cost associated with a specific route, based on the route number provided by the user.



```
/*---- SELECT A ROUTE ----*/
start:-

select_route.

select_route:-
    display_all_routes,

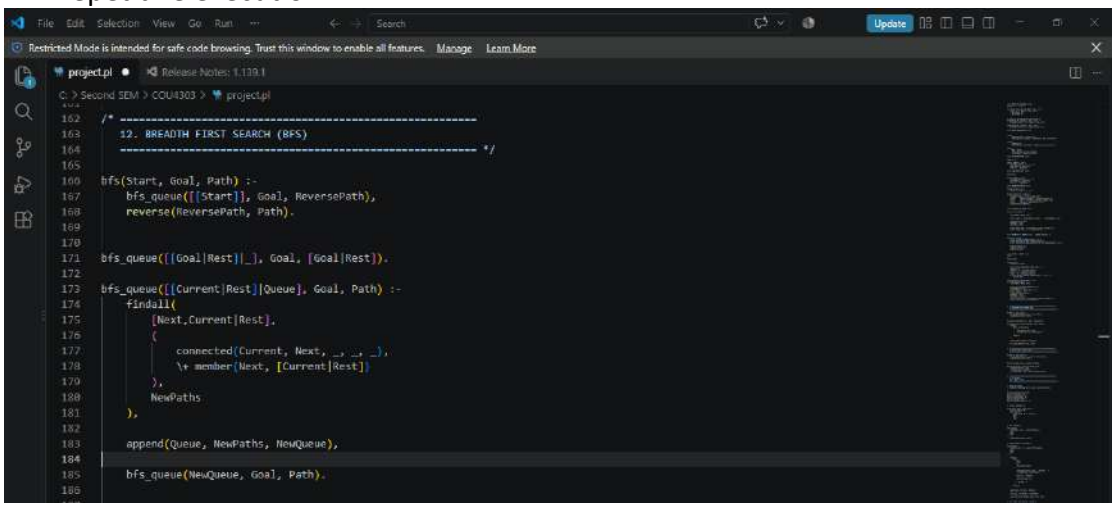
    write('Select available route (1-3): '),
    read(Choice),
    (Choice = 1 -> selected_route(1);
    Choice = 2 -> selected_route(2);
    Choice = 3 -> selected_route(3);
    writeln('Invalid choice! Please enter 1, 2 or 3.'),
        select_route
    ).
/*---- DISPLAY SELECTED ROUTE ----*/
selected_route(Number):-
    route(Number, Name, Path),

    write('-----SELECTED ROUTE-----'),nl,
    write(Number),nl,nl,
    write('Route: '),write(Name),nl,nl,
    write('Journey: '),nl,
    display_details(Path),nl,
    time(Path, Time),
    cost(Path, Cost),nl,
    write('Total Time: '),write(Time),write(' minutes'),nl,
    write('Total Cost:Rs.'),write(Cost).
```

- Rechecking the code corrections and formatting the code to ensure the output is presented in an organized manner.

5. I V C M Bandara : 724655849

- The code has been designed to find a path using the BFS algorithm and to handle repetitive execution.



```
/* ------------------------------------------------------
   12. BREADTH FIRST SEARCH (BFS)
   ------------------------------------------------------ */

bfs(Start, Goal, Path) :-
    bfs_queue([[Start]], Goal, ReversePath),
    reverse(ReversePath, Path).

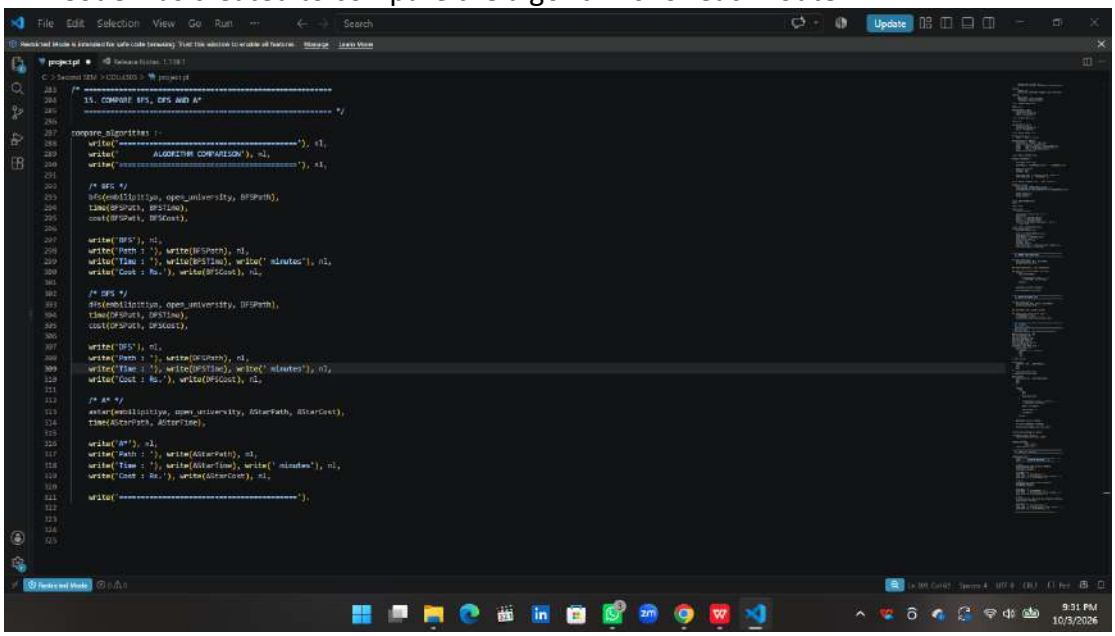

bfs_queue([[Goal|Rest]|_], Goal, [Goal|Rest]).

bfs_queue([[Current|Rest]|Queue], Goal, Path) :-
    findall(
        [Next,Current|Rest],
        (
            connected(Current, Next, _, _, _),
            \+ member(Next, [Current|Rest])
        ),
        NewPaths
    ),

    append(Queue, NewPaths, NewQueue),

    bfs_queue(NewQueue, Goal, Path).
```

- Code was created to compare the algorithms for each route.



```
/* ------------------------------------------------------
   13. COMPARE BFS, DFS AND A*
   ------------------------------------------------------ */

compare_algorithms :-
    write('=========================================='), nl,
    write('       ALGORITHM COMPARISON'), nl,
    write('=========================================='), nl,

    /* BFS */
    bfs(embilipitiya, open_university, BFSPath),
    time(BFSPath, BFSTime),
    cost(BFSPath, BFSCost),

    write('BFS'), nl,
    write('Path : '), write(BFSPath), nl,
    write('Time : '), write(BFSTime), write(' minutes'), nl,
    write('Cost : Rs.'), write(BFSCost), nl,

    /* DFS */
    dfs(embilipitiya, open_university, DFSPath),
    time(DFSPath, DFSTime),
    cost(DFSPath, DFSCost),

    write('DFS'), nl,
    write('Path : '), write(DFSPath), nl,
    write('Time : '), write(DFSTime), write(' minutes'), nl,
    write('Cost : Rs.'), write(DFSCost), nl,

    /* A* */
    astar(embilipitiya, open_university, AStarPath, AStarCost),
    time(AStarPath, AStarTime),

    write('A*'), nl,
    write('Path : '), write(AStarPath), nl,
    write('Time : '), write(AStarTime), write(' minutes'), nl,
    write('Cost : Rs.'), write(AStarCost), nl,

    write('==========================================').
```

6. K L K P Liyanage : 524679572

- The code has been designed to find a path using the DFS algorithm and to handle repetitive execution.



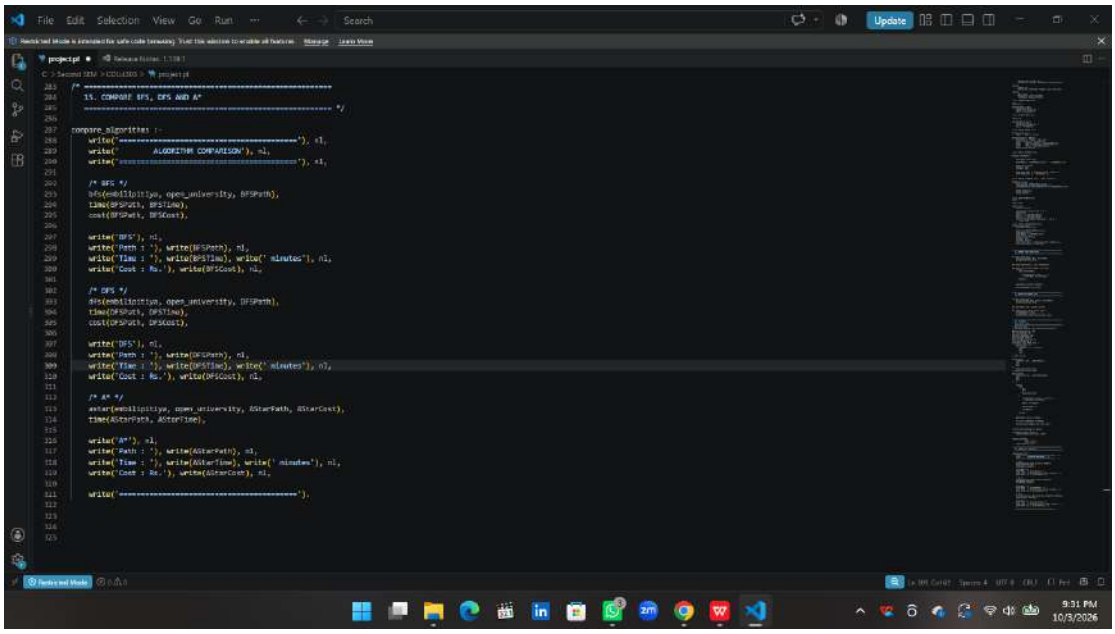
```
/* ------------------------------------------------------
   13. DEPTH FIRST SEARCH (DFS)
   ------------------------------------------------------ */

dfs(Start, Goal, Path) :-
    dfs_search(Start, Goal, [Start], ReversePath),
    reverse(ReversePath, Path).


dfs_search(Goal, Goal, Visited, Visited).

dfs_search(Current, Goal, Visited, Path) :-
    connected(Current, Next, _, _, _),
    \+ member(Next, Visited),
    dfs_search(Next, Goal, [Next|Visited], Path).
```

- Code was created to compare the algorithms for each route.



```
/* ------------------------------------------------------
   14. COMPARE BFS, DFS AND A*
   ------------------------------------------------------ */

compare_algorithms :-
    write('=========================================='), nl,
    write('       ALGORITHM COMPARISON'), nl,
    write('=========================================='), nl,

    /* BFS */
    bfs(embilipitiya, open_university, BFSPath),
    time(BFSPath, BFSTime),
    cost(BFSPath, BFSCost),

    write('BFS'), nl,
    write('Path : '), write(BFSPath), nl,
    write('Time : '), write(BFSTime), write(' minutes'), nl,
    write('Cost : Rs.'), write(BFSCost), nl,

    /* DFS */
    dfs(embilipitiya, open_university, DFSPath),
    time(DFSPath, DFSTime),
    cost(DFSPath, DFSCost),

    write('DFS'), nl,
    write('Path : '), write(DFSPath), nl,
    write('Time : '), write(DFSTime), write(' minutes'), nl,
    write('Cost : Rs.'), write(DFSCost), nl,

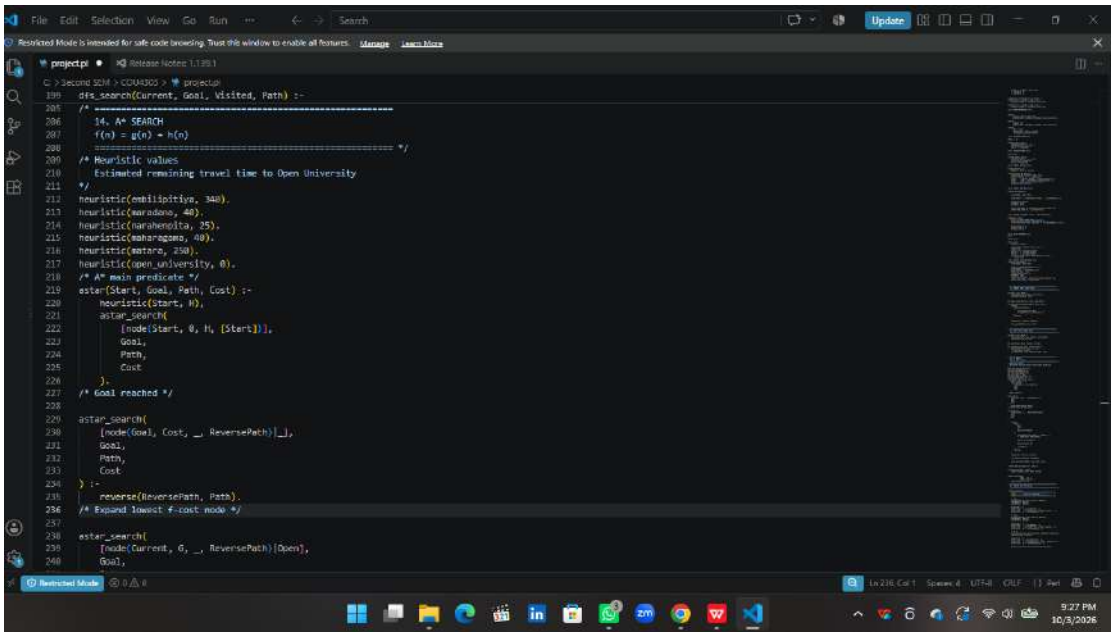
    /* A* */
    astar(embilipitiya, open_university, AStarPath, AStarCost),
    time(AStarPath, AStarTime),

    write('A*'), nl,
    write('Path : '), write(AStarPath), nl,
    write('Time : '), write(AStarTime), write(' minutes'), nl,
    write('Cost : Rs.'), write(AStarCost), nl,

    write('==========================================').
```

7. R S Warnakulasooriya : 624683193

- Writing code to find an efficient path using the A* algorithm.

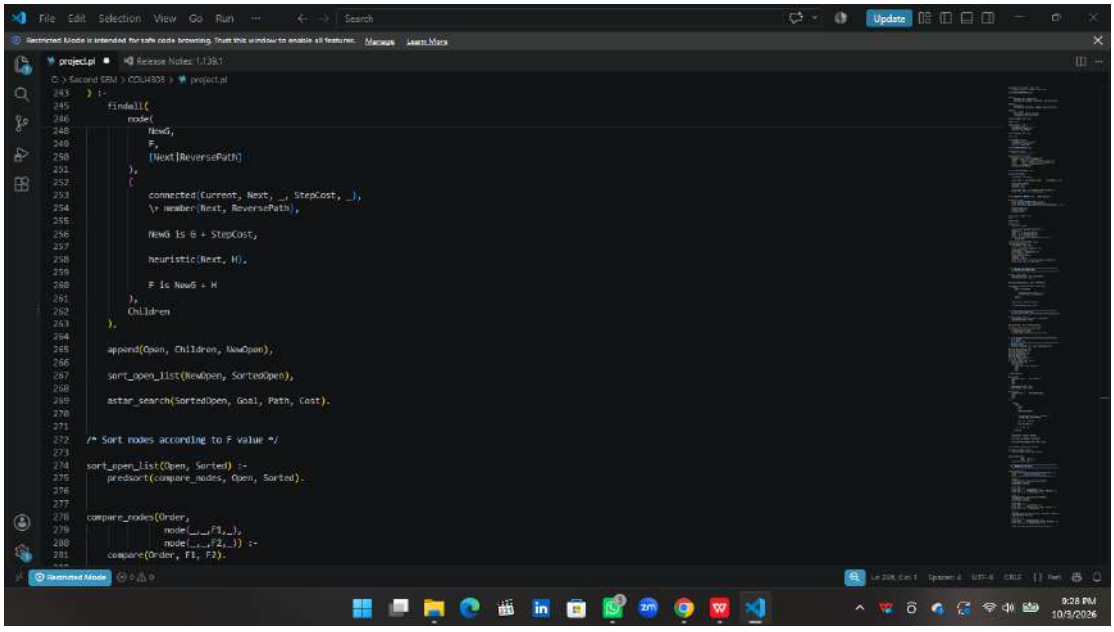


```
dfs_search(Current, Goal, Visited, Path) :-

/* ============================================================
   14. A* SEARCH
   f(n) = g(n) + h(n)
   ============================================================ */
/* Heuristic values
   Estimated remaining travel time to Open University
*/
heuristic(embilipitiya, 340).
heuristic(waradana, 40).
heuristic(narahenpita, 25).
heuristic(maharagama, 40).
heuristic(matara, 250).
heuristic(open_university, 0).
/* A* main predicate */
astar(Start, Goal, Path, Cost) :-
    heuristic(Start, H),
    astar_search(
        [node(Start, 0, H, [Start])],
        Goal,
        Path,
        Cost
    ).
/* Goal reached */

astar_search(
    [node(Goal, Cost, _, ReversePath)|_],
    Goal,
    Path,
    Cost
) :-
    reverse(ReversePath, Path).
/* Expand lowest F-cost node */

astar_search(
    [node(Current, G, _, ReversePath)|Open],
    Goal,
```



```
) :-
    findall(
        node(
            NewG,
            F,
            [Next|ReversePath]
        ),
        (
            connected(Current, Next, _, StepCost, _),
            \+ member(Next, ReversePath),

            NewG is G + StepCost,

            heuristic(Next, H),

            F is NewG + H
        ),
        Children
    ),

    append(Open, Children, NewOpen),

    sort_open_list(NewOpen, SortedOpen),

    astar_search(SortedOpen, Goal, Path, Cost).


/* Sort nodes according to F value */

sort_open_list(Open, Sorted) :-
    predsort(compare_nodes, Open, Sorted).


compare_nodes(Order,
              node(_,_,F1,_),
              node(_,_,F2,_)) :-
    compare(Order, F1, F2).
```